1 Estimator Infrastructure: WLS Seeding, State Initialisation, and Pivot Management

The Kalman filter of the filtering document is a *local* linear estimator. Every update assumes the state is close to the linearisation point. Every dynamic state (ambiguity, per-satellite ionosphere, receiver code bias, satellite bias, or inter-system offset; the state document) needs a value and a covariance before it can be updated. This document covers the machinery that provides them:

- the weighted-least-squares (WLS) epoch solve, which places the receiver close enough for the linearisation to hold (Section 1.1);
- the first-epoch seeds formed from residual differences, which give each new state a physically reasonable prior (Section 1.2);
- the between-satellite pivot selection and disturbance logic, which keep the single-difference (SD) reference stable across an arc (Section 1.3).

None of this is filtering. It is what makes the filter's local-linear assumptions valid at the first epoch and after every discontinuity. The first-epoch seeds are pure functions of the observation equation (the shared observation equation of the Foundations document), written in `measurement/Seeds.cpp`. The single-receiver model applies them to its `StateManager` (`measurement/MeasurementModelSeeds.cpp`) and the network estimator to a station's parameter set (`network/assembly/LocalBirth.cpp`).

The lossless covariance re-referencing (the $\boldsymbol{T}\,\boldsymbol{P}\,\boldsymbol{T}^{\mathrm{T}}$ S-transform that moves a group onto a new pivot) is *out of scope* here. It is derived in the differencing document and implemented once in `include/filter/CovarianceReReference.hpp`, which both the single-receiver and the network handover paths call. This section notes where the pivot logic calls it.

1.1 The WLS epoch estimator: receiver-clock seed and post-gap re-seed

Before the filter can run, the receiver position must be close enough to the truth that the geometry Jacobian linearised about it is valid to below a decimetre. Broadcast pseudoranges are sufficient for this. The engine runs a GPS-only, code-only iterated Gauss–Newton solve over the core state

$$\boldsymbol{\xi} = \begin{bmatrix} \boldsymbol{r}^{\mathrm{T}} & c\,\mathrm{d}t_r & \mathrm{ZWD} \end{bmatrix}^{\mathrm{T}} \in \mathbb{R}^5, \tag{1}$$

with $\boldsymbol{r} \in \mathbb{R}^3$ the ECEF receiver position, $c\,\mathrm{d}t_r$ the receiver clock in metres, and ZWD the zenith wet delay (`src/core/math/WlsEstimator.cpp`). At least `kCoreSize` $= 5$ code observations are required before the solve is attempted; the iteration cap is `kMaxIter` $= 5$.

For each code measurement i the prefit residual removes every modelled term and the current state's contribution,

$$\widetilde{z}_i = P_i - \left[\, P_i^{\mathrm{mod}} + (\rho_i - \rho_i^0) + c\,\mathrm{d}t_r + m_w(i)\,\mathrm{ZWD} \,\right], \tag{2}$$

where P_i^{mod} is the modelled pseudorange, $\rho_i - \rho_i^0$ the change in geometric range relative to the range at the iteration's linearisation point, and $m_w(i)$ the wet mapping function (`WlsEstimator::predictObs`, evaluated once per row from the design-row loop of `WlsEstimator::estimate`). The

geometric range is *re-evaluated every iteration* about the current position estimate, so each iteration is a Gauss–Newton relinearisation.

The design row for satellite i is the negated line-of-sight stacked with the clock and wet-mapping partials, assembled in the same loop:

$$\boldsymbol{A}_i = \begin{bmatrix} -\boldsymbol{e}_i^{\mathrm{T}} & 1 & m_w(i) \end{bmatrix} = \begin{bmatrix} -e_x & -e_y & -e_z & 1 & m_w(i) \end{bmatrix}, \tag{3}$$

with $\boldsymbol{e}_i$ the receiver-to-satellite unit vector (the same line-of-sight that drives the observation geometry). Stacking all rows into $\boldsymbol{A}$, the prefit residuals into $\widetilde{z}$, and the inverse measurement variances onto the diagonal weight $\boldsymbol{W}$ (clamped from below by `kMinVariance` $= 10^{-6}$ in that same loop), the normal-equations step is the boxed Gauss–Newton update

$$\boxed{\Delta\boldsymbol{\xi} = \left(\boldsymbol{A}^{\mathrm{T}}\boldsymbol{W}\,\boldsymbol{A}\right)^{-1}\boldsymbol{A}^{\mathrm{T}}\boldsymbol{W}\,\widetilde{z},} \tag{4}$$

applied once per Gauss–Newton iteration of `WlsEstimator::estimate`. Iteration terminates on a position increment $\|\Delta\boldsymbol{r}\| <$ `kTol` $= 0.1$ m or at `kMaxIter`. The normal matrix is guarded against rank deficiency: if

$$\det\!\left(\boldsymbol{A}^{\mathrm{T}}\boldsymbol{W}\,\boldsymbol{A}\right) \le 10^{-6} \tag{5}$$

the solve aborts and the epoch is not seeded (the determinant guard immediately before the step). This protects against degenerate single-constellation, nearly coplanar geometry, which would otherwise produce a very large position step.

On convergence the estimator returns both the state $\boldsymbol{\xi} = [\boldsymbol{r}^{\mathrm{T}},\ c\,\mathrm{d}t_r,\ \mathrm{ZWD}]^{\mathrm{T}}$ and its formal covariance, the inverse normal matrix

$$\boldsymbol{P}_{\boldsymbol{\xi}} = \left(\boldsymbol{A}^{\mathrm{T}}\boldsymbol{W}\,\boldsymbol{A}\right)^{-1} \tag{6}$$

(the result assembly at the end of `WlsEstimator::estimate`). The filter uses only the *clock* from this result. `Application::handleWlsSeeding` writes $c\,\mathrm{d}t_r$ into the receiver-clock state when one of three triggers fires: the first epoch, a data gap, or a clock *jump* of at least `kClockJumpThreshSec` $= 50\,\mu$s (about 15 km) between the WLS and filter clock estimates. In practice the jump is the most common trigger.

The position, the zenith wet delay, and the covariance (6) are computed but are not used as filter seeds. The position prior is the RINEX-header `approx_position` plus the antenna offset. The WLS position re-seeds the filter only on a kinematic (non-IMU) cold start after a data gap (`Application::handleGap`). The ZWD estimate goes only to a trace-CSV column, and nothing in the filter reads the formal covariance. The whole seeding path is skipped under BSSD, where the filter has no receiver-clock state (`handleWlsSeeding` returns early on `!hasClock()`).

The main use of the clock seed is *re-seeding after a gap*. The receiver clock is the most volatile state across a data gap, because an unsteered oscillator can drift freely while no measurements constrain it. After a gap the WLS clock estimate resets $c\,\mathrm{d}t_r$ before the phase filter resumes. Otherwise the filter would correct a stale clock prior through a series of large innovations.

1.2 First-epoch state seeding by residual differencing

The WLS solve of Section 1.1 seeds the receiver clock and, on a kinematic cold start, the position. It does not touch the *differential* states: inter-system bias, phase clock, uncombined receiver code bias, per-satellite ionosphere, and carrier ambiguity. Each of these is created empty and must be seeded from the measurement residuals. The common idea (`src/core/measurement/Seeds.cpp`) is that each state is what remains in a particular *difference* of residuals after all modelled terms are

removed. The seed is that difference, taken as a median over satellites for robustness. The shared prefit residual is

$$\tilde{z}^i_{r,s} = \Phi/P^i_{r,s} - \big(\text{modelled} + c\,\mathrm{d}t_r + m_w\,\mathrm{ZWD} + \mathrm{ISB} + g_{\mathrm{trop}} + \alpha_s \widehat{I}^i + \beta_s b_r + b^i\big), \tag{7}$$

the observation minus every term the model can already account for, evaluated at the current state (`rawResidual` in `measurement/Seeds.cpp`). The residual is the equation of the shared observation equation built with a mask of the terms known when the state is born, and its terms are subtracted in a fixed order: the modelled value, the receiver clock, the wet troposphere, the inter-system bias, the gradient projection $g_{\mathrm{trop}} = m_g(g_N \cos\mathrm{az} + g_E \sin\mathrm{az})$, the slant ionosphere (with the sign + on a code row and − on a phase row), the receiver code bias and the satellite code bias. The four *differential* seeds (a)–(d) below are differences of (7), the ambiguity seed (e) is the phase residual with its own mask, and a sixth pre-seed (f) anchors the receiver clock itself.

(a) Inter-system bias. Each non-GPS constellation g carries a receiver clock offset relative to the GPS time scale (the inter-system offset state). The GPS clock is the datum, and the WLS step has already put it into $c\,\mathrm{d}t_r$. The ISB of g is therefore the median code residual of the constellation minus the median GPS residual:

$$\boxed{\mathrm{ISB}_g = \mathrm{med}_{i\in g}\ \tilde{z}^i - \mathrm{med}_{i\in\mathrm{GPS}}\ \tilde{z}^i} \tag{8}$$

(`seedIsb`, applied by `MeasurementModel::initializeInterSystemBiases` and by the station birth). Medians are used throughout, so one gross code outlier cannot spoil the seed. A guard requires at least one GPS code observation, the time reference, at the seeding epoch. If GPS is absent the seed reports itself unsettled and *is tried again at the next epoch*. It does not mark the ISB states as initialised against an arbitrary datum. Without the guard, whichever constellation is acquired first would become the implicit zero, and a GPS satellite rising later would then carry the whole offset.

(b) Phase clock. In the uncombined model the phase observations see a receiver clock that differs from the code clock by the receiver phase–code hardware delay (the phase-clock state). Its seed is the median over satellites carrying *both* a code and a phase observation of the code-minus-phase residual:

$$\delta_\Phi = \mathrm{med}_i\ \big(\tilde{z}^i_P - \tilde{z}^i_\Phi\big) \tag{9}$$

(`seedPhaseClock`, applied by `MeasurementModel::initializePhaseClockOffset`). The covariance of the seed is set from the code noise. It is tighter than the default initial standard deviation, and it does not depend on the number n of differences in the median (the covariance seeding in `initializePhaseClockOffset`):

$$\sigma_{\delta_\Phi} = \max\Big(\tfrac{\sigma_P}{\sqrt{n}},\ \sigma_P\Big), \qquad \boldsymbol{P}(\delta_\Phi, \delta_\Phi) = \sigma^2_{\delta_\Phi}, \tag{10}$$

The corresponding row and column of $\boldsymbol{P}$ are set to zero first, so the seeded state starts uncorrelated. Since $\sigma_P/\sqrt{n} \le \sigma_P$ for every $n \ge 1$, the max always selects σ_P, and the seeded variance is σ^2_P whatever the value of n. The seed is floored at the noise of a single code measurement and does not improve as $1/\sqrt{n}$. The network estimator has no phase clock and does not form this seed.

(c) Uncombined receiver code bias / IFB. In the uncombined PPP-RTK model each non-reference code band carries a receiver differential code bias (DCB) or inter-frequency bias (IFB) state relative to the constellation's BIA clock-reference band (the code-bias state). Forming the residual difference between band j and the reference band, removing the differential ionospheric contribution (the two bands scale the same slant ionosphere by different α), and dividing by the difference of receiver-bias Jacobian coefficients gives

$$\boxed{\widehat{b}_{r,j} = \frac{\Delta\widetilde{z}_j - \Delta(\alpha_j\,\widehat{I})}{\Delta\beta_j} = \frac{(\widetilde{z}_j - \widetilde{z}_{\mathrm{ref}}) - (\alpha_j - \alpha_{\mathrm{ref}})\,\widehat{I}^i}{\beta_j - \beta_{\mathrm{ref}}}} \tag{11}$$

(`seedCodeBias`), median-aggregated over satellites and subject to the same $|\widehat{b}| > 200\,\mathrm{m}$ reject as the ionosphere seed below; $\widehat{I}^i$ is the current value of the satellite's slant-ionosphere state. After a warm resume of the network estimator, a key that the installed linearisation points cover takes the installed value in place of the median (`StationParams::rx_cb_restore`). The Jacobian coefficient β_s depends on the bias parameterisation (`ReceiverBiasResolver::resolveReceiverBias`): for a pure IFB state it is the unit coefficient $\beta = 1$, whereas for a geometry-free / ionosphere-free DCB state against reference bands of frequencies f_1, f_2 it is

$$\mu_1 = 1, \quad \mu_2 = \frac{f_1^2}{f_2^2}, \quad \Delta\mu = \mu_1 - \mu_2, \qquad \beta_j = \begin{cases} \mu_1/\Delta\mu & j = \text{band 1} \\ \mu_2/\Delta\mu & j = \text{band 2,} \end{cases} \tag{12}$$

guarded by $|\Delta\mu| \geq 10^{-12}$ to avoid dividing by a near-degenerate frequency separation.

For GLONASS a third branch runs before both cases. Under `enable_glonass_ifb` each FDMA band (G1, G2) has one bias state *per FCN slot* k with $\beta = 1$ (`StateKeys::makeGlonassPerSlotIfbKey`). The per-slot carrier $f = f_{\mathrm{base}} + k\,\Delta f$ makes the receiver group delay depend on the slot, which one state per band cannot represent. The central channel $k = 0$ is the datum and has no state. The CDMA bands (G3/G4/G6) use the per-band treatment above. The branch runs before the clock-reference lookup because GLONASS usually has no clock-reference pair in the `.BIA` product, and the routine would otherwise return before reaching any IFB logic. With the flag off, a non-reference GLONASS band has no IFB state (the per-slot and non-reference-band branches of `resolveReceiverBias`).

(d) Per-satellite slant ionosphere. Each tracked satellite's slant ionospheric delay (the per-satellite iono state) is recovered from the geometry-free combination of two code bands. With ionospheric scale factors $\alpha_s = f_{\mathrm{ref}}^2/f_s^2$ referenced to GPS L1 (the iono-scaling convention), the seed is

$$\boxed{\widehat{I}^i = \frac{(P_1 - P_1^{\mathrm{mod}}) - (P_2 - P_2^{\mathrm{mod}})}{\alpha_1 - \alpha_2}} \tag{13}$$

(`seedIonosphere`), where P_k^{mod} is the modelled pseudorange on band k. Two guards apply. The denominator must satisfy $|\alpha_1 - \alpha_2| \geq 10^{-6}$. The seed is rejected unless it is finite and $|\widehat{I}^i| < 200\,\mathrm{m}$. A slant ionospheric delay above 200 m is not physical. It indicates mis-paired bands or a gross code outlier, and must not be stored in a per-satellite state.

Two BIA-coverage gates also apply, so that the differenced code is consistent in its biases. The first is *presence*: a `.BIA` entry exists on the band, with the zero-valued BeiDou C2I/C6I entries treated as missing. The second is the coverage *application* check (`bias_coverage_requires_apply`). It skips the seed when an entry is loaded but not *applied* to ($P -$ mod), because the raw hardware bias is then still present. The network estimator uses presence only. The application check is needed

because the failure it prevents, a zero-valued BeiDou entry that injects 30–60 m, is well below the 200 m reject and would pass it.

(e) Carrier ambiguity. The float ambiguity that subsequently feeds integer resolution (the ambiguity-resolution document) is the part of the *phase* residual that remains after every other modelled and estimated term is removed (`seedAmbiguity`, the routine that every phase-seeding call site uses),

$$\widehat{N}^i_{r,s} = \Phi^i_{r,s} - \text{modelled} - \left(c\,\mathrm{d}t_r + \delta_\Phi\right) - m_w\,\mathrm{ZWD} - \mathrm{ISB} - g_{\mathrm{trop}} + \alpha_s\,\widehat{I}^i, \tag{14}$$

This is the phase observation minus its model, the receiver clock *and* the phase clock δ_Φ (seed (b)), the wet delay, the inter-system bias, and the tropospheric gradient, plus the slant ionospheric phase advance $+\alpha_s\,\widehat{I}^i$ (seed (d)). The phase is advanced by the same α_s that delays the code, so the iono term has the opposite sign to (7). The phase clock and the slant iono are removed because they are estimated states in the uncombined model. One term cannot be removed this way: a receiver clock that is still zero on a network cold start. Pre-seed (f) handles that case.

The seeded ambiguity therefore contains the integer cycle count plus the remaining receiver and satellite phase biases. AR recovers the integer part once the satellite phase bias is supplied and the receiver phase bias cancels under between-satellite differencing.

(f) Cold receiver-clock pre-seed (network path). Seeds (a)–(e) assume the receiver clock has already been solved. That holds on the PPP path, where the WLS clock seed of Section 1.1 runs first. The network estimator does not run the WLS seed, so nothing solves the receiver clock before the ambiguity of (e) is seeded. `seedAmbiguity` then subtracts a clock term of zero, and the whole free-running receiver-clock offset goes into every cold ambiguity seed.

The opt-in `seedColdClock` (`network.cold_clock_seed`, loader default off, armed in both reference configurations) fixes this. It writes the median GPS code residual into the clock parameter, the median that ISB seed (a) forms for the time reference. The seed overwrites only a *cold* clock. Any $|x(\text{clock})| >$ `kColdClockEpsilonM` $= 10^{-6}$ m is treated as already solved and is left unchanged. When the epoch has no GPS code observation the seed reports no reference, so it tries again at the next epoch and does *not* mark the clock as seeded. This matches the GPS-reference contract of seed (a). A station's birth applies it once, after the slant ionospheres and before the ambiguities (`seedColdClockOnce`, `network/assembly/LocalBirth.cpp`); the single-receiver model applies it in the same place of its state pre-pass (`MeasurementModel::preseedColdReceiverClock`).

1.3 BSSD pivot management

Between-satellite single differencing (BSSD, the BSSD document) re-expresses each constellation's ambiguities relative to a designated *pivot* satellite p_g. This cancels the receiver phase bias and makes the SD ambiguities integer. The choice of pivot matters. A poor pivot degrades every SD row that references it, and an unstable one forces costly re-referencing handovers. `src/core/filter/PivotManager.cpp` selects and maintains the pivot. The baseline double-difference path uses the similar `src/core/filter/DdPivotManager.cpp`.

1.3.1 Lexicographic candidate ranking

Candidates are ordered *lexicographically*: first by a signal-coverage tier, then by a continuous score within the tier. Coverage comes first because the pivot must track every signal that any non-pivot satellite tracks. A non-pivot signal that the pivot lacks has no reference and is dropped. *Any*

full-coverage candidate therefore ranks above *any* partial-coverage candidate, whatever their scores (the lexicographic comparator in `PivotManager::buildEpochMaps`). Candidate a beats b if and only if its phase-signal count is higher, or the counts are equal and its score is higher.

Within a coverage tier the tie-break score combines float-convergence and geometry (`PivotManager::scorePivotCandidate`):

$$\boxed{\text{score}(i) = -\min\!\big(1,\ \sqrt{v_{\max}(i)}\,\big) + \frac{\text{el}(i)}{90} - \tfrac{1}{2}\,\mathbb{1}[\text{el}(i) < 20^\circ]\,,} \tag{15}$$

where $v_{\max}(i)$ is the worst (largest) float-ambiguity variance among the satellite's tracked phase signals and $\text{el}(i)$ is its elevation in degrees.

The first term penalises a pivot that has not converged. On handover the new pivot's variance is added to every SD state in its group. This is the diagonal of the S-transform, $\boldsymbol{P}'(i,i) = \boldsymbol{P}(i,i) + \boldsymbol{P}(p_g,p_g) - 2\boldsymbol{P}(i,p_g)$, implemented in `include/filter/CovarianceReReference.hpp`. A pivot whose own state is still settling therefore passes its uncertainty to the whole constellation. The penalty saturates at -1, equivalent to one metre of 1σ ambiguity uncertainty. A candidate with *no* state yet receives a flat -1 (the negative-`max_amb_variance` sentinel branch of the score). On the first BSSD epoch every candidate in a tier has this sentinel, so the score reduces to elevation and no special first-election branch is needed.

The el/90 term prefers high-elevation pivots. The $-\frac{1}{2}$ penalty below 20° discourages a low pivot, whose tropospheric and multipath residuals exceed the formal stochastic model however well its float state has converged.

Both terms only *rank* candidates that have already passed a hard elevation gate. A satellite below `pivot_min_el` (`getBssdPivotMinElevation`) cannot be elected at all: not in the freshest-peer scan, not in the online pick, and not in the sticky-retention test, all three in `PivotManager::selectPivotCandidates`. The $-\frac{1}{2}$ penalty therefore only breaks ties among the survivors. When *no* satellite passes the mask (a code-only float constellation), the gate is bypassed. The pick reverts to the precomputed best or highest-elevation satellite, so the candidate set is never empty (the fallback from `pickExclusionAwareBest` to the precomputed `best_sat` / `highest_sat`).

Sticky retention is the code's *elevation hysteresis*, and it prevents the pivot changing back and forth between epochs. It uses the same *absolute* gate and does not compare the incumbent with a challenger. The incumbent is kept whenever it still passes `pivot_min_el`, is not excluded, and stays within the toe spread of Section 1.3.2 (the sticky branch of `selectPivotCandidates`). No score is evaluated on that branch, so a better challenger cannot displace a qualifying incumbent. The score ranking runs only after the incumbent has *failed* the gate.

1.3.2 Ephemeris-freshness guardrail

A pivot with a stale broadcast ephemeris puts a step into *every* SD row that references it. Its extrapolated orbit and clock drift away from those of the fresher satellites it is differenced against. This was observed in production: a Galileo pivot held a one-hour-old I/NAV record while the other satellites' records were minutes old, and the whole SD solution was corrupted. The guardrail is gated on the *spread* of ages. A candidate is barred from pivot election when its absolute ephemeris age $|t_{\text{obs}} - t_{\text{oe}}|$ exceeds the constellation's freshest qualified peer by more than

$$\texttt{kPivotToeSpreadMaxS} = 1200\ \text{s} = 20\ \text{min} \tag{16}$$

(the `freshestToeAgeAbs` scan in `PivotManager::selectPivotCandidates`, applied both at the online pick and at the sticky-retention test). Because the gate is on the *spread* and not on an

absolute age, the ranking is unchanged when all candidates are about equally fresh. Only a candidate that is much staler than the rest is displaced. The barred satellite can still be used as a non-pivot. The freshest satellite always passes, since its difference from itself is zero, so the candidate set is never empty.

1.3.3 Pivot disturbance detection and handover

In BSSD the pivot's own SD ambiguity is zero with zero variance, because the pivot *is* the reference: $x(p_g) = 0$ and $\boldsymbol{P}(p_g, p_g) = 0$ exactly. Any departure from that state means the filter has perturbed the reference, through process noise or an update, and the SD states are no longer consistent. `isPivotDisturbed` flags this when, for any of the pivot's phase signals,

$$\boxed{\left|x(p_g)\right| > 10^{-6} \quad \text{or} \quad \boldsymbol{P}(p_g, p_g) > 10^{-12}} \tag{17}$$

(`PivotManager::isPivotDisturbed`). The two thresholds are tolerances for numerical zero, not tuned gates. 10^{-6} m on the value and $10^{-12}\,\mathrm{m}^2$ on the variance catch any real leakage onto the pivot and tolerate floating-point round-off in the re-reference arithmetic. The predict step must not create a false disturbance, so the pivot's process noise is suppressed during prediction and $\boldsymbol{P}(p_g, p_g)$ stays at zero (the freeze-state / $\boldsymbol{Q}$-suppression mechanics). Without the suppression, every epoch's $\boldsymbol{Q}$ would trip (17) and force an unnecessary handover.

A handover fires when the incumbent is disturbed or when the ranking elects a different satellite. The outgoing pivot is then classified. In a *clean* handover the old pivot's state is invertible and consistent. It can be re-referenced without loss, and the RTS smoother can later invert it. A *forced* or disturbed handover cannot be inverted cleanly and marks a discontinuity.

In both cases `CovReReference::applyBssd` in `include/filter/CovarianceReReference.hpp` re-references the covariance in place. It subtracts the new pivot's row and column from each non-pivot member of the group, then sets the new pivot's row and column to zero. This is the one shared implementation of the $\boldsymbol{T}\boldsymbol{P}\boldsymbol{T}^{\mathrm{T}}$ transform derived in the differencing document.

The baseline DD path has a different structure. It negates the new-pivot row and column and subtracts from the others, which is a different S-transform, and then renames the affected ambiguity states with `StateManager::renameAmbiguity` (`DdPivotManager::handleHandover`). The two paths share only the positive-definiteness guard applied after the transform (`enforcePsd`). The single-receiver user path runs the transform without the guard. At user scale the groups are small and handovers are rare, so floating-point error in $\boldsymbol{T}\boldsymbol{P}\boldsymbol{T}^{\mathrm{T}}$ does not accumulate.